

RPI Pico-ETH-CH9121

Code Documentation

Contact: Ibrahim Elbir (ibrahimelbir97@gmail.com)

ch9121.py

1. `__init__(self, uart)`

Purpose: Initializes the CH9121 object with default settings.

Parameters:

uart: A UART object used for communication with the CH9121 module.

Attributes:

MODE: Operation mode (0: TCP Server, 1: TCP Client, 2: UDP Server, 3: UDP Client).

GATEWAY, TARGET_IP, LOCAL_IP, SUBNET_MASK: Network configuration details such as gateway IP, target IP, local IP, and subnet mask.

LOCAL_PORT1, LOCAL_PORT2, TARGET_PORT: Ports for local and target communication.

BAUD_RATE: Baud rate for UART communication.

CFG: GPIO pin used to configure the CH9121.

RST: GPIO pin used to reset the CH9121.

Usage: This method is called when creating an instance of the CH9121 class. It sets up the initial configuration and pin assignments.

2. **enter_config(self)**

Purpose: Puts the CH9121 module into configuration mode.

Usage: Call this method before sending configuration commands to the CH9121. It resets the module and pulls the configuration pin low to enter config mode.

3. **exit_config(self)**

Purpose: Exits configuration mode and saves the settings.

Usage: After setting the desired configuration, call this method to save the settings and return the module to normal operation. It sends specific commands to exit config mode and sets the CFG pin high.

4. **set_mode(self, mode)**

Purpose: Sets the operation mode of the CH9121.

Parameters:

mode: An integer representing the mode (0 for TCP Server, 1 for TCP Client, etc.).

Usage: Call this method to change the mode of the CH9121. The mode value is sent to the module as a command.

5. **set_local_ip(self, local_ip)**

Purpose: Sets the local IP address of the CH9121.

Parameters:

local_ip: A tuple representing the local IP address (e.g., (192, 168, 1, 227)).

Usage: Use this method to set the IP address of the device. The IP address is sent as a command to the module.

6. **set_subnet_mask(self, subnet_mask)**

Purpose: Sets the subnet mask for the CH9121.

Parameters:

subnet_mask: A tuple representing the subnet mask (e.g., (255, 255, 255, 0)).

Usage: Call this method to configure the subnet mask. The subnet mask is sent to the module as a command.

7. **set_gateway(self, gateway)**

Purpose: Sets the gateway IP address for the CH9121.

Parameters:

gateway: A tuple representing the gateway IP address (e.g., (192, 168, 1, 1)).

Usage: Use this method to set the gateway IP address. The gateway IP is sent to the module as a command.

8. **set_local_port1(self, local_port1)**

Purpose: Sets the local port number for communication.

Parameters:

local_port1: An integer representing the local port (e.g., 502).

Usage: Call this method to set the port number the CH9121 will use for local communication. The port number is sent to the module as a command.

9. **set_target_ip(self, target_ip)**

Purpose: Sets the target IP address for communication.

Parameters:

target_ip: A tuple representing the target IP address (e.g., (192, 168, 1, 227)).

Usage: Use this method to set the target IP address for communication. The target IP is sent to the module as a command.

10. **set_target_port(self, target_port)**

Purpose: Sets the target port number for communication.

Parameters:

target_port: An integer representing the target port (e.g., 502).

Usage: Call this method to set the port number for the target device the CH9121 will communicate with. The port number is sent to the module as a command.

11. **set_baud_rate(self, baud_rate)**

Purpose: Sets the UART baud rate for communication with the CH9121.

Parameters:

baud_rate: An integer representing the baud rate (e.g., 115200).

Usage: Use this method to set the UART baud rate. The baud rate is sent to the module as a command.

registers.py

1. `__init__(self)`

Purpose: Initializes the Registers object with default settings.

Attributes:

INPUT_REGISTERS: A list of 20 integers initialized to 0, representing the input registers.

HOLDING_REGISTERS: A list of 30 integers initialized to 0, representing the holding registers.

Usage: This method is called when an instance of the Registers class is created. It initializes the input and holding registers with default values.

2. `read_hr(self, order)`

Purpose: Reads a value from the holding registers.

Parameters:

order: An integer representing the index of the holding register to read.

Returns: The value stored in the holding register at the specified index.

Usage: Call this method to retrieve the value of a specific holding register.

3. `read_ir(self, order)`

Purpose: Reads a value from the input registers.

Parameters:

order: An integer representing the index of the input register to read.

Returns: The value stored in the input register at the specified index.

Usage: Call this method to retrieve the value of a specific input register.

4. **write_hr(self, order, data)**

Purpose: Writes a value to the holding registers.

Parameters:

order: An integer representing the index of the holding register to write to.

data: The value to write into the holding register at the given index.

Usage: Use this method to update the value of a specific holding register.

5. **write_ir(self, order, data)**

Purpose: Writes a value to the input registers.

Parameters:

order: An integer representing the index of the input register to write to.

data: The value to write into the input register at the given index.

Usage: Use this method to update the value of a specific input register.

6. **length_hr(self)**

Purpose: Retrieves the length of the holding registers.

Returns: An integer representing the number of holding registers.

Usage: Call this method to get the total number of holding registers available.

7. **length_ir(self)**

Purpose: Retrieves the length of the input registers.

Returns: An integer representing the number of input registers.

Usage: Call this method to get the total number of input registers available.

modbus.py

1. `__init__(self, slave_id=1, uart=None, port=502)`

Purpose: Initializes the Modbus object with the provided parameters, including slave ID, UART, and port.

Attributes:

slave_id: The ID of the Modbus slave; default is 1.

uart: The UART object for serial communication; default is None.

port: The TCP port for the Modbus server; default is 502.

Usage: This method is called when an instance of the Modbus class is created. It initializes the Modbus slave ID, UART, and port.

2. `checkNoneParams(self, kwargs)`

Purpose: Validates that none of the provided parameters are None.

Parameters:

kwargs: Keyword arguments representing parameters to check.

Returns: Raises a ModbusValidationError if any parameters are None.

Usage: This method ensures that critical parameters like slave_id and uart are not None.

3. `check_ip_range(self, ip)`

Purpose: Validates that the IP address bytes are within the valid range (0-255).

Parameters:

ip: List of integers representing the IP address.

Returns: True if the IP address is valid; False otherwise.

Usage: Call this method to validate the IP address before updating the configuration.

4. **check_slave_id_range(self, slave_id)**

Purpose: Validates that the slave ID is within the valid range (1-255).

Parameters:

slave_id: Integer representing the Modbus slave ID.

Returns: True if the slave ID is valid; False otherwise.

Usage: Call this method to validate the slave ID before updating the configuration.

5. **check_port_range(self, port)**

Purpose: Validates that the TCP port is within the valid range (0-65535).

Parameters:

port: Integer representing the TCP port.

Returns: True if the port is valid; False otherwise.

Usage: Call this method to validate the port number before updating the configuration.

6. **start_server(self)**

Purpose: Continuously listens for incoming Modbus requests on the UART and processes them.

Usage: Call this method to start the Modbus server that handles incoming requests over UART.

7. **handle_client(self, conn)**

Purpose: Asynchronously handles communication with a Modbus client, processing requests and sending responses.

Parameters:

conn: The connection object representing the client.

Usage: This method is used within an asyncio loop to manage client communication for Modbus over TCP.

8. **async_recv(self, conn, buffer_size)**

Purpose: Asynchronously receives data from a Modbus client.

Parameters:

conn: The connection object representing the client.

buffer_size: The maximum amount of data to receive.

Returns: The data received from the client.

Usage: This method is used to receive data asynchronously from a Modbus client connection.

9. **process_modbus_request(self, data)**

Purpose: Processes an incoming Modbus request and generates an appropriate response.

Parameters:

data: The byte data representing the Modbus request.

Returns: The byte data representing the Modbus response or an exception response.

Usage: Call this method to process incoming Modbus requests and generate responses.

10. `handle_read_holding_registers(self, pdu)`

Purpose: Handles the "Read Holding Registers" Modbus function code (0x03).

Parameters:

pdu: The Protocol Data Unit (PDU) containing the request details.

Returns: The response PDU with the requested register values or an exception response.

Usage: This method is used to process requests for reading holding registers.

11. `handle_read_input_registers(self, pdu)`

Purpose: Handles the "Read Input Registers" Modbus function code (0x04).

Parameters:

pdu: The Protocol Data Unit (PDU) containing the request details.

Returns: The response PDU with the requested register values or an exception response.

Usage: This method is used to process requests for reading input registers.

12. `handle_write_single_register(self, pdu)`

Purpose: Handles the "Write Single Register" Modbus function code (0x06).

Parameters:

pdu: The Protocol Data Unit (PDU) containing the request details.

Returns: The response PDU confirming the write operation or an exception response.

Usage: This method is used to process requests for writing a single holding register.

13. `handle_write_multiple_register(self, pdu)`

Purpose: Handles the "Write Multiple Registers" Modbus function code (0x10).

Parameters:

pdu: The Protocol Data Unit (PDU) containing the request details.

Returns: The response PDU confirming the write operation or an exception response.

Usage: This method is used to process requests for writing multiple holding registers.

14. `generate_exception_response(self, pdu, exception_code)`

Purpose: Generates an exception response PDU for the given exception code.

Parameters:

pdu: The original request PDU that caused the exception.

exception_code: The Modbus exception code to include in the response.

Returns: The response PDU with the exception code.

Usage: Call this method to generate an appropriate exception response for a failed Modbus request.

main.py

Script Overview:

1. Key Components:

Imports and Initial Setup: Required modules and classes like logger, Sensor, UART, Pin, REGS, asyncio, file_handler, CH9121, Modbus, and struct are imported.

Sensor and Reset Pin: Initializes a sensor object and sets up a reset pin for the CH9121 Ethernet module.

2. Task Functions:

modbus_server_task(modbus): Continuously checks for Modbus requests on UART and processes them, sending back responses if available.

sensor_read_task(sensor): Periodically reads temperature and humidity data from holding registers, applies calibration, and writes the results to input registers.

3. Main Initialization and Configuration:

UART and CH9121 Configuration: Sets up the UART and configures the CH9121 module with network settings such as IP, subnet mask, gateway, and port.

Configuration Handling: Reads or creates a config.json file with necessary settings and initializes holding registers with values from this configuration.

Modbus Initialization: Initializes the Modbus object with the configured slave ID and UART settings.

Task Creation and Execution: Creates tasks for the Modbus server and sensor reading, running them concurrently using asyncio.

Usage:

The script initializes and configures the necessary hardware and software components, then runs tasks to handle Modbus communication and sensor data processing. It can be used to run a Modbus TCP server on a device while simultaneously collecting and calibrating sensor data.